

Федеральное государственное автономное образовательное учреждение
высшего образования «**Национальный исследовательский университет
ИТМО**»

Факультет Программной Инженерии и Компьютерной Техники

**Лабораторная работа №3
по дисциплине «Вычислительная математика»**

Вариант: **10**

Преподаватель:
Наумова Надежда Александровна

Выполнил: Ровкова Анастасия Сергеевна
Группа: P3216

Санкт-Петербург, 2026 г

Цель работы

Найти приближенное значение определенного интеграла с требуемой точностью различными численными методами, выполнить программную реализацию методов на языке программирования SCALA.

10	$\int_2^4 (x^3 - 3x^2 + 7x - 10)dx$
----	-------------------------------------

Порядок выполнения работы + Вычисление интеграла

Вар n10

1. Точное значение

$$\begin{aligned}\int_2^4 (x^3 - 3x^2 + 7x - 10)dx &= \left. \frac{x^4}{4} - x^3 + \frac{7x^2}{2} - 10x \right|_2^4 = \\ &= \cancel{64} - \cancel{64} + 56 - 40 - 4 + 8 - 14 + 20 = \underline{26}\end{aligned}$$

2. Ньютона - Котеса n = 6

$$n = 6 : C_6^0 = C_6^6 = \frac{41(6-a)}{840}$$

$$C_6^1 = C_6^5 = \frac{216(6-a)}{840}$$

$$C_6^2 = C_6^4 = \frac{27(6-a)}{840}$$

$$C_6^3 = \frac{272(6-a)}{840}$$

$$h = \frac{b-a}{n} = \frac{2}{6} = \frac{1}{3}$$

$$x_0 = a = 2$$

$$x_1 = a + h = 2\frac{1}{3}$$

$$x_2 = a + 2h = 2\frac{2}{3}$$

$$x_3 = a + 3h = 3$$

$$x_4 = a + 4h = 3\frac{1}{3}$$

$$x_5 = a + 5h = 3\frac{2}{3}$$

$$x_6 = a + 6h = 4$$

$$y_0 = 0$$

$$y_1 = 2,7$$

$$y_2 = 6,288$$

$$y_3 = 11$$

$$y_4 = 17,03$$

$$y_5 = 24,613$$

$$y_6 = 34$$

$$\begin{aligned}I &\approx \frac{1}{420} \cdot (41y_0 + 216y_1 + 27y_2 + 272y_3 + 27y_4 + 216y_5 + 41y_6) = \\ &\approx 25,9885\end{aligned}$$

3. По формулам ср. прямоугол., трапеций и Симпсона
при $n = 10$

$$h = \frac{b-a}{n} = \frac{2}{10} = 0,2$$

1) Ср. прямоугольники

$$x_{0,5} = 2,1$$

$$x_{1,5} = 2,3$$

$$x_{2,5} = 2,5$$

$$x_{3,5} = 2,7$$

$$x_{4,5} = 2,9$$

$$x_{5,5} = 3,1$$

$$x_{6,5} = 3,3$$

$$x_{7,5} = 3,5$$

$$x_{8,5} = 3,7$$

$$x_{9,5} = 3,9$$

$$y_{0,5} = 0,721$$

$$y_{1,5} = 2,397$$

$$y_{2,5} = 4,375$$

$$y_{3,5} = 6,713$$

$$y_{4,5} = 9,459$$

$$y_{5,5} = 12,661$$

$$y_{6,5} = 16,367$$

$$y_{7,5} = 20,625$$

$$y_{8,5} = 25,483$$

$$y_{9,5} = 30,989$$

$$I \approx 0,2 \cdot (\sum y) = 25,96$$

2) Трапеции

$$x_0 = a = 2$$

$$x_1 = a + h = 2,2$$

$$x_2 = a + 2h = 2,4$$

$$x_3 = a + 3h = 2,6$$

$$x_4 = a + 4h = 2,8$$

$$x_5 = a + 5h = 3$$

$$x_6 = a + 6h = 3,2$$

$$y_0 = 0$$

$$y_1 = 1,528$$

$$y_2 = 3,344$$

$$y_3 = 5,496$$

$$y_4 = 8,032$$

$$y_5 = 11$$

$$y_6 = 14,448$$

$$x_7 = a + 7h = 3,4$$

$$y_7 = 18,424$$

$$x_8 = a + 8h = 3,6$$

$$y_8 = 22,976$$

$$x_9 = a + 9h = 3,8$$

$$y_9 = 28,152$$

$$x_{10} = a + 10h = 4$$

$$y_{10} = 34$$

$$I = \frac{h}{2} (y_0 + y_n + 2 \cdot (y_1 + \dots + y_{n-1})) \approx 26,08$$

3) Симпсона

x_i, y_i возьмем из

$$I = \frac{h}{3} (y_0 + y_n + 4 \sum y_{\text{неч}} + 2 \sum y_{\text{чет}}) =$$

$$= 26$$

4. Сравним результаты + Погрешности

Ньютона - Котеса : $\Delta I = |26 - 25,9885| = 0,115$

Ср. прямоугольники : $\Delta I = |26 - 25,96| = 0,04$

Трапеции : $\Delta I = |26 - 26,08| = 0,08$

Симпсона : $\Delta I = |26 - 26| = 0$

Рабочие формулы используемых методов.

Формула Ньютона-Котеса порядка n:

$$\int_a^b f(x) dx \approx \int_a^b L_n(x) dx = \sum_{i=0}^n f(x_i) c_n^i$$

Метод средних прямоугольников

При $h_i = h = \frac{b-a}{n} = \text{const}$:

$$\int_a^b f(x) dx = h \sum_{i=1}^n f(x_{i-1/2})$$

Метод трапеций

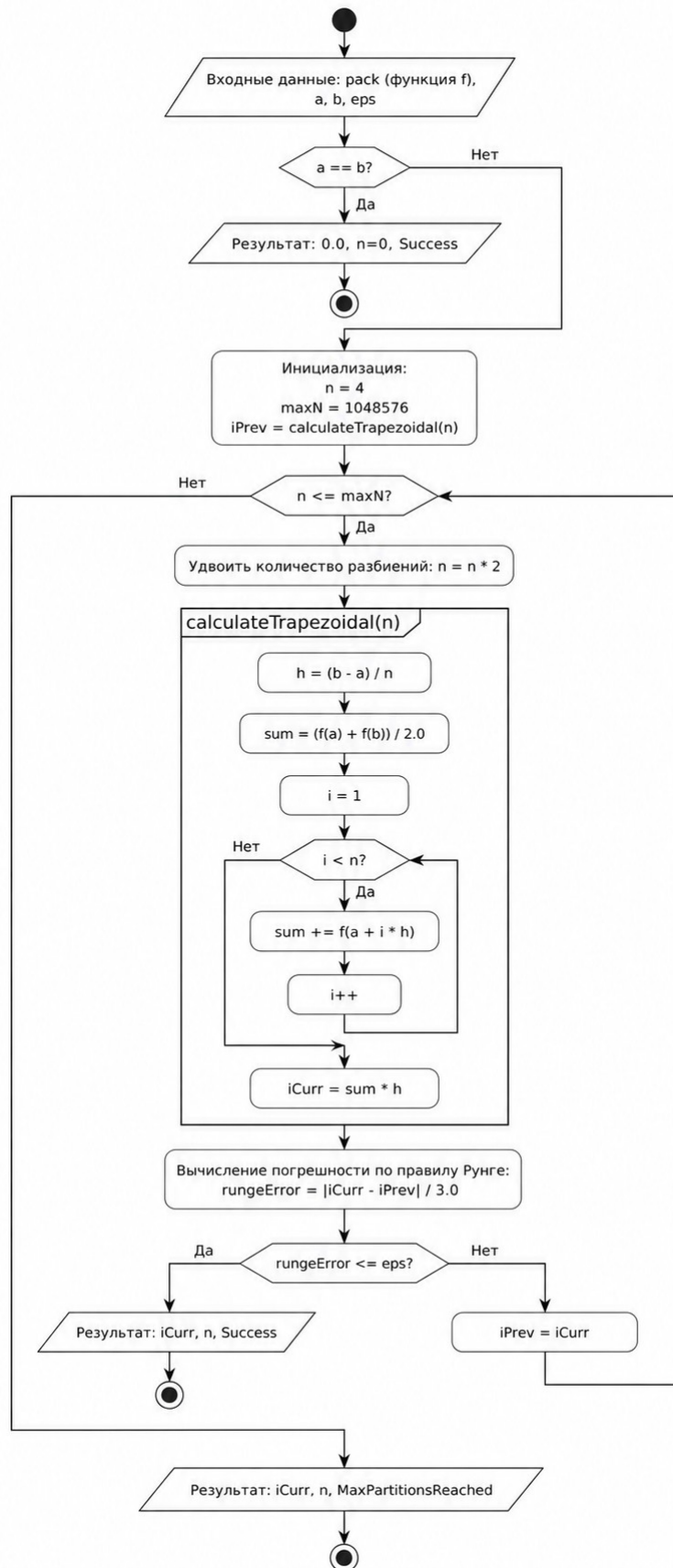
$$\int_a^b f(x) dx = \frac{h}{2} \cdot \left(y_0 + y_n + 2 \sum_{i=1}^{n-1} y_i \right)$$

Формула Симпсона

$$\int_a^b f(x) = \frac{h}{3} [(y_0 + 4(y_1 + y_3 + \dots + y_{n-1}) + 2(y_2 + y_4 + \dots + y_{n-2}) + y_n)]$$

Блок схемы и листинги методов

Алгоритм метода трапеций (Trapezoid Method)



package solver.integrals.methods

```
import solver.integrals.integrals.{IntegralsPack, IntegrationMethod,
IntegralsResult}
import solver.core.Message

class TrapezoidMethod extends IntegrationMethod {
  override protected def calculateAlgorithm(pack: IntegralsPack, a: Double, b:
Double, eps: Double): IntegralsResult = {
    if (a == b) return IntegralsResult(0.0, 0, Message.Success)

    def calculateTrapezoidal(n: Int): Double = {
      val h = (b - a) / n
      var sum = (pack.f(a) + pack.f(b)) / 2.0

      for (i <- 1 until n) {
        sum += pack.f(a + i * h)
      }
      sum * h
    }

    var n = 4
    var iPrev = calculateTrapezoidal(n)
    var iCurr = 0.0
    var rungeError = Double.MaxValue
    val maxN = 1048576

    while (n <= maxN) {
      n *= 2
      iCurr = calculateTrapezoidal(n)

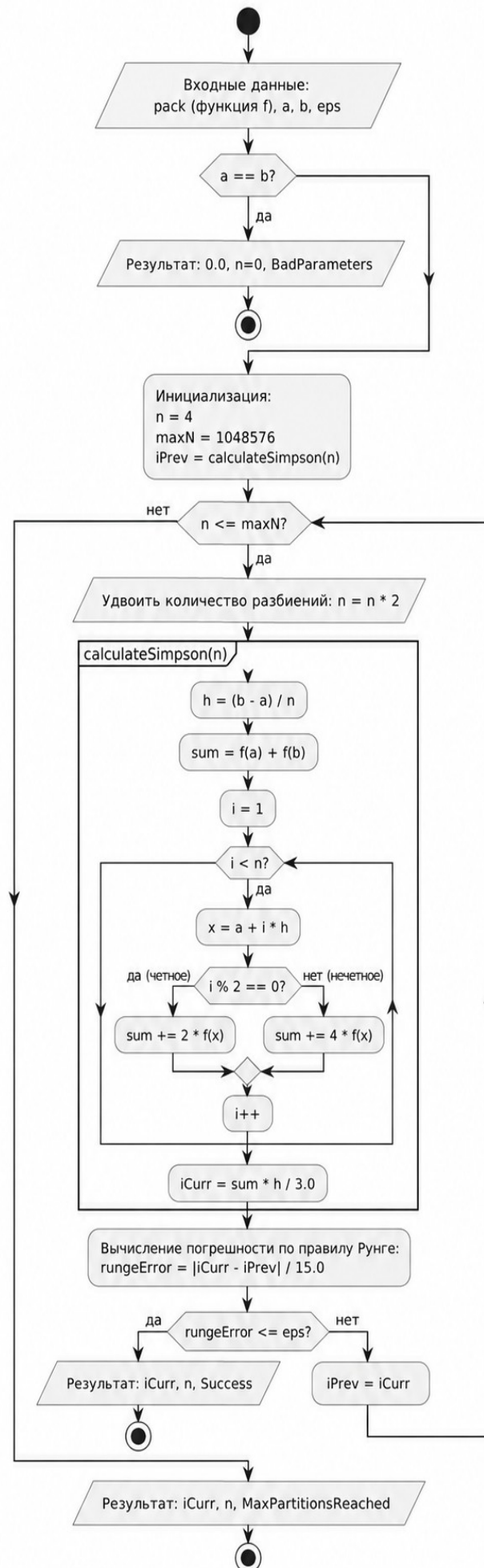
      rungeError = math.abs(iCurr - iPrev) / 3.0

      if (rungeError <= eps) {
        return IntegralsResult(iCurr, n, Message.Success)
      }

      iPrev = iCurr
    }

    IntegralsResult(iCurr, n, Message.MaxPartitionsReached)
  }
}
```


Алгоритм метода Симпсона (Simpson's Method)



package solver.integrals.methods

```
import solver.integrals.integrals.{IntegralsPack, IntegrationMethod,
IntegralsResult}
import solver.core.Message

class SimpsonMethod extends IntegrationMethod {

  override protected def calculateAlgorithm(pack: IntegralsPack, a: Double, b:
Double, eps: Double): IntegralsResult = {

    if (a == b) {
      return IntegralsResult(0.0, 0, Message.BadParameters)
    }

    def calculateSimpson(n: Int): Double = {
      val h = (b - a) / n
      var sum = pack.f(a) + pack.f(b)

      for (i <- 1 until n) {
        val x = a + i * h
        if (i % 2 == 0) {
          sum += 2 * pack.f(x)
        } else {
          sum += 4 * pack.f(x)
        }
      }
      sum * h / 3.0
    }

    var n = 4
    var iPrev = calculateSimpson(n)
    var iCurr = 0.0
    var rungeError = Double.MaxValue
    val maxN = 1048576

    while (n <= maxN) {
      n *= 2
      iCurr = calculateSimpson(n)

      rungeError = math.abs(iCurr - iPrev) / 15.0

      if (rungeError <= eps) {
        return IntegralsResult(iCurr, n, Message.Success)
      }
    }
  }
}
```

```
}
```

```
    iPrev = iCurr
```

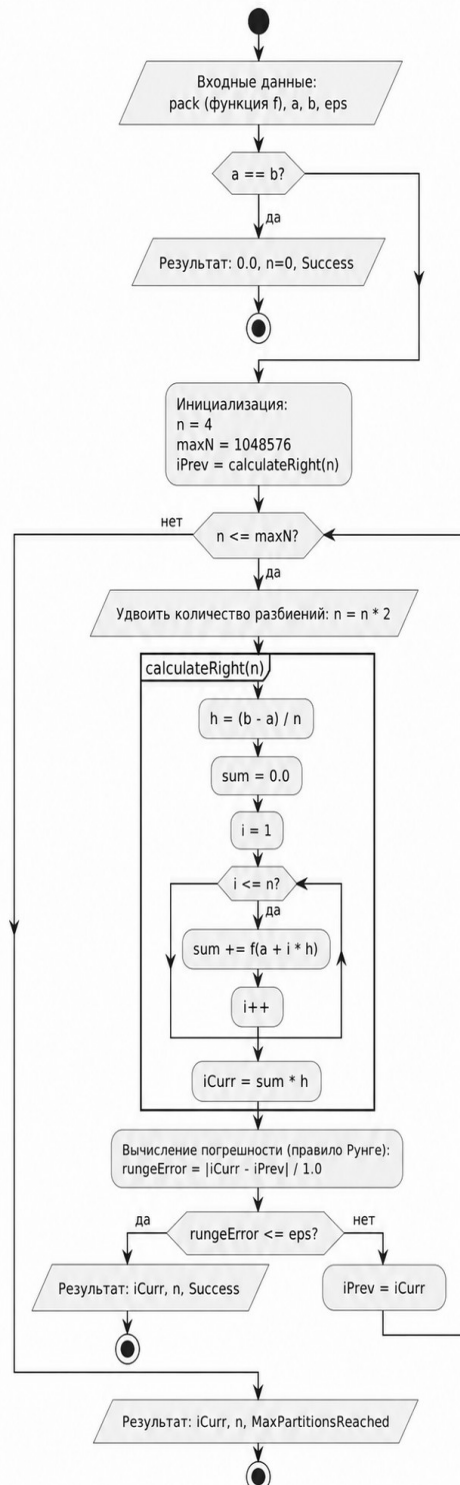
```
}
```

```
IntegralsResult(iCurr, n, Message.MaxPartitionsReached)
```

```
}
```

```
}
```

Алгоритм метода правых прямоугольников (Rectangle Method Right)



```

package solver.integrals.methods

import solver.integrals.integrals.{IntegralsPack, IntegrationMethod,
IntegralsResult}
import solver.core.Message

class RectangleMethodRight extends IntegrationMethod {
  override protected def calculateAlgorithm(pack: IntegralsPack, a: Double, b:
Double, eps: Double): IntegralsResult = {
    if (a == b) return IntegralsResult(0.0, 0, Message.Success)

    def calculateRight(n: Int): Double = {
      val h = (b - a) / n
      var sum = 0.0

      for (i <- 1 to n) {
        sum += pack.f(a + i * h)
      }
      sum * h
    }

    var n = 4
    var iPrev = calculateRight(n)
    var iCurr = 0.0
    val maxN = 1048576

    while (n <= maxN) {
      n *= 2
      iCurr = calculateRight(n)

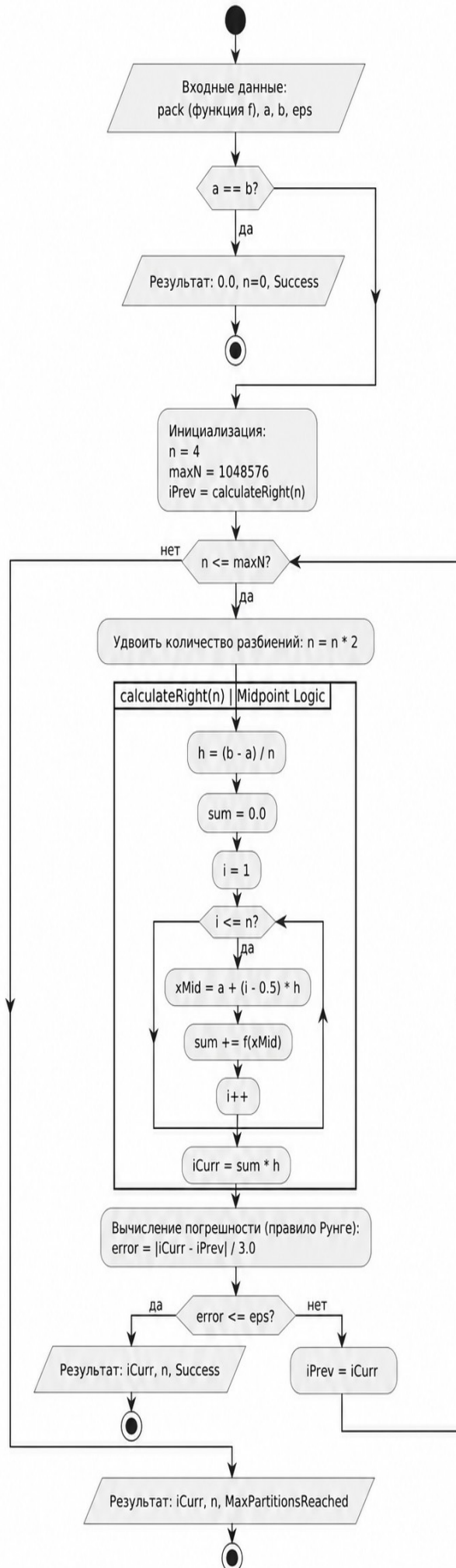
      if (math.abs(iCurr - iPrev) / 1.0 <= eps) {
        return IntegralsResult(iCurr, n, Message.Success)
      }

      iPrev = iCurr
    }

    IntegralsResult(iCurr, n, Message.MaxPartitionsReached)
  }
}

```

Метод средних прямоугольников (Rectangle Method Mid)



```
package solver.integrals.methods
```

```
import solver.integrals.integrals.{IntegralsPack, IntegrationMethod,  
IntegralsResult}
```

```
import solver.core.Message
```

```
class RectangleMethodMid extends IntegrationMethod {  
  override protected def calculateAlgorithm(pack: IntegralsPack, a: Double, b:  
Double, eps: Double): IntegralsResult = {  
    if (a == b) return IntegralsResult(0.0, 0, Message.Success)
```

```
    def calculateRight(n: Int): Double = {  
      val h = (b - a) / n  
      var sum = 0.0
```

```
      for (i <- 1 to n) {  
        val xMid = a + (i - 0.5) * h  
        sum += pack.f(xMid)  
      }  
      sum * h  
    }
```

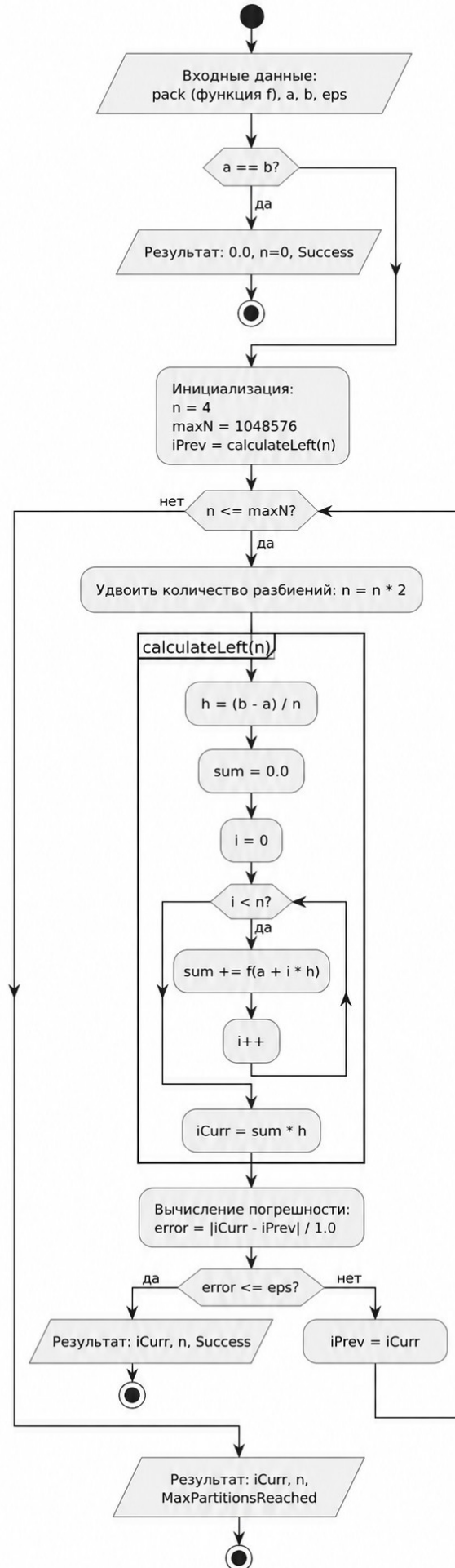
```
    var n = 4  
    var iPrev = calculateRight(n)  
    var iCurr = 0.0  
    val maxN = 1048576
```

```
    while (n <= maxN) {  
      n *= 2  
      iCurr = calculateRight(n)  
  
      if (math.abs(iCurr - iPrev) / 3.0 <= eps) {  
        return IntegralsResult(iCurr, n, Message.Success)  
      }
```

```
      iPrev = iCurr  
    }
```

```
    IntegralsResult(iCurr, n, Message.MaxPartitionsReached)  
  }  
}
```

Метод левых прямоугольников (Rectangle Method Left)



```
package solver.integrals.methods
```

```
import solver.integrals.integrals.{IntegralsPack, IntegrationMethod,  
IntegralsResult}
```

```
import solver.core.Message
```

```
class RectangleMethodLeft extends IntegrationMethod {  
  override protected def calculateAlgorithm(pack: IntegralsPack, a: Double, b:  
Double, eps: Double): IntegralsResult = {  
    if (a == b) return IntegralsResult(0.0, 0, Message.Success)
```

```
    def calculateLeft(n: Int): Double = {  
      val h = (b - a) / n  
      var sum = 0.0
```

```
      for (i <- 0 until n) {  
        sum += pack.f(a + i * h)  
      }  
      sum * h  
    }
```

```
    var n = 4  
    var iPrev = calculateLeft(n)  
    var iCurr = 0.0  
    val maxN = 1048576
```

```
    while (n <= maxN) {  
      n *= 2  
      iCurr = calculateLeft(n)
```

```
      if (math.abs(iCurr - iPrev) / 1.0 <= eps) {  
        return IntegralsResult(iCurr, n, Message.Success)  
      }
```

```
      iPrev = iCurr  
    }
```

```
    IntegralsResult(iCurr, n, Message.MaxPartitionsReached)  
  }  
}
```


Листинг программы

https://github.com/Naswek/ITMO_labs/commit/607fd2a23d98be7ba24fef1121ffd08745f95fbc —

иммутабельная ссылка на КОММИТ

Результаты выполнения программы при различных исходных данных.

Лаб 2 (Уравнения)

Лаб 3 (Интегралы)

Численное интегрирование

Настройки интеграла

Выберите функцию:

$\int (-x^3 - x^2 - 2x + 1) dx$

Выберите метод:

Метод Симпсона

Нижний предел (a)

0

Верхний предел (b)

2

Точность (epsilon)

0.001

Вычислить интеграл

Результат:

Значение интеграла: -8.6669921875

Число разбиений (n): 128

Статус: Успешно

Численное интегрирование

Настройки интеграла

Выберите функцию:

$\int (5x^3 - 2x^2 + 3x - 15) dx$



Выберите метод:

Метод Симпсона



Нижний предел (a)

-2

Верхний предел (b)

0

Точность (epsilon)

0.00001

Вычислить интеграл

Результат:

Значение интеграла: -61.33333873748779

Число разбиений (n): 2048

Статус: Успешно

Численное интегрирование

Настройки интеграла

Выберите функцию:

$\int 1 / \sqrt{x} \, dx$

Выберите метод:

Метод правых треугольников

Нижний предел (a)

-2

Верхний предел (b)

0

Точность (epsilon)

0.00001

Вычислить интеграл

Результат:

Интеграл не существует (расходится)

Вывод:

В работе изучены численные методы нахождения примерного значения интеграла. Почти совпадение с точным значением интеграла, подтверждает корректность реализованных алгоритмов на языке программирования SCALA.

